MINI PROJECT REPORT

STUDENT PERFORMANCE PREDICTION USING DEEP NEURAL NETWORKS

A Data-Driven Approach for Early Identification of Academically At-Risk Students

Submitted by

Name: PRIYANKA G

Register Number: 711524BCS130

Class & Section: III- CSE- B

Department of Computer Science and Engineering

ABSTRACT

Education institutions generate large amounts of data about students, including attendance, internal assessment marks, study habits, family background, and previous results. When this data is analysed only after final examinations, teachers lose the opportunity to support students who are struggling. Student Performance Prediction Using Deep Neural Networks proposes a data-driven system that uses such records to estimate a student's likely academic outcome, such as a pass or fail result, a grade band, or a final score, while there is still time to intervene.

The proposed approach uses a Deep Neural Network (DNN), a feed-forward network with several hidden layers, to learn relationships among academic, behavioural, and demographic attributes. Records are collected from a labelled student dataset, cleaned, encoded, scaled, and divided into training, validation, and testing sets. The DNN is trained to predict the performance category, and its behaviour is controlled using techniques such as dropout, batch normalization, and early stopping. The trained model can then be connected to a simple interface where an educator enters student details and receives a predicted outcome with a probability estimate.

The project is intended as a decision-support tool for teachers and counsellors rather than a replacement for human judgement. Its reliability depends on data quality, the representativeness of the training records, and fairness across student groups. Evaluation can be performed using accuracy, precision, recall, F1-score, and a confusion matrix for classification, or mean absolute error and root mean squared error for score prediction. The system may support early support and timely counselling, while further validation is required before real institutional use.

KEYWORDS

Deep Learning, Deep Neural Network, Student Performance Prediction, Educational Data Mining, Classification, Early Warning System, Feature Engineering, Dropout Regularization.

TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives

4. Scope of the Project

5. Existing System

6. Proposed System

7. System Requirements

8. Dataset Description and Preparation

9. Methodology

10. System Architecture

11. Deep Neural Network Model Design
12. Implementation Plan
13. Evaluation Metrics
14. Expected Results
15. Advantages
16. Limitations
17. Applications
18. Future Enhancements
19. Conclusion
20. References

INTRODUCTION

Academic performance is influenced by many factors, including prior marks, attendance, study time, parental support, health, motivation, and access to learning resources. Weak performance often develops gradually, and early signs may appear in assignments, class tests, or attendance records long before final results are published. Identifying these signs early is an important part of student support and academic planning.

In conventional practice, teachers review marks and attendance manually and rely on personal experience to identify students who may need help. Although teacher insight is valuable, this process becomes difficult when class sizes are large and many factors interact. Statistical methods can help, but they often assume simple relationships. Deep learning can automatically learn non-linear combinations of features from examples rather than relying entirely on manually written rules.

A deep-neural-network-based performance prediction system accepts a student's record, applies the same preprocessing used in training, and predicts a class or score learned from historical data. For example, the output may be Pass or Fail, or a grade band such as Low, Medium, or High. The exact target depends on the dataset used. The output is a prediction, not a certainty; it should be used to start a supportive conversation and should be reviewed by qualified educators.

PROBLEM STATEMENT

Manual identification of students who are likely to perform poorly is slow, inconsistent, and often happens too late. Teachers may not be able to examine many attributes for every student, and the same factors may affect different students in different ways. The problem addressed by this project is to develop a prediction system that uses historical student records to estimate performance categories accurately and efficiently. The system should provide a clear result, be straightforward to use, and be evaluated on records that were not used during training.

OBJECTIVES

- To study the use of deep learning and educational data mining for predicting student performance.
- To prepare a labelled dataset of student records with academic, behavioural, and demographic attributes.
- To preprocess data by handling missing values, encoding categorical features, and scaling numerical features.
- To design and train a Deep Neural Network with suitable hidden layers, activation functions, and regularization.
- To predict the performance category or final score of a student from the available attributes.
- To evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- To create a simple user interface for entering student details and viewing the predicted outcome.
- To document limitations and encourage educator review and fair, responsible use of the predictions.

SCOPE OF THE PROJECT

The mini-project focuses on predicting student performance from a finite set of attributes available in the training dataset. It may begin with a binary target such as Pass or Fail, or a small number of grade bands, before being extended to score prediction or multiple subjects. The system can be developed as a Python prototype and connected to a lightweight web interface. It does not identify the causes of poor performance, recommend disciplinary action, or replace the judgement of teachers and counsellors.

The system's performance should be tested on records that were not used for training and, where possible, on data from a different academic year or institution. The scope should clearly state which attributes and targets are supported, because a model cannot reliably predict for populations or features that differ greatly from its training data.

EXISTING SYSTEM

Existing approaches include manual review of marks and attendance by teachers, counselling based on personal observation, periodic progress reports, and statistical or traditional machine-learning models such as linear regression, decision trees, and support vector machines. Human insight can consider personal context and remains essential in difficult cases. However, manual review requires time and experience, and simple models may not capture complex, non-linear interactions among many features.

Drawbacks of the Existing System

- Manual review may vary between teachers and is difficult to scale to large classes.
- Problems are often noticed only after poor results are published, which leaves little time for support.

- Simple statistical models may not capture complex interactions among academic and behavioural factors.
- Many factors, such as attendance and study habits, act together, making rule-based prediction unreliable.
- A model trained on one institution's data may not perform equally well on another.

PROPOSED SYSTEM

The proposed system uses a Deep Neural Network to learn patterns directly from student records. During training, the model receives records with known outcomes and adjusts its weights to reduce prediction error. During prediction, a new record is passed through the same preprocessing pipeline, and the trained model returns class probabilities. The class with the highest probability is displayed along with a confidence estimate.

A typical workflow includes data entry, validation, preprocessing, model inference, result display, and optional feedback collection. Regularization techniques such as dropout and early stopping are used because educational datasets are often small and prone to overfitting. The final results must be measured on a separate test set.

Main Modules

- Data Acquisition: accepts student records from a dataset file or user entry.
- Preprocessing: handles missing values, encodes categorical attributes, and scales numerical attributes.
- Model Training: trains a DNN with hidden layers using labelled student records.
- Prediction: generates probabilities for each performance class and selects the highest-scoring class.
- Result Display: presents the predicted outcome and, if appropriate, the model's confidence score.
- Evaluation: compares predictions with the true outcomes and calculates performance metrics.

SYSTEM REQUIREMENTS

Hardware Requirements

- Computer or laptop with a modern multi-core processor.
- At least 8 GB RAM recommended; more memory is helpful for larger datasets.
- Adequate storage for the dataset, source code, and trained model.
- GPU optional; the dataset is small, so a CPU is usually sufficient for training.

Software Requirements

- Operating System: Windows, Linux, or macOS.
- Programming Language: Python 3.
- Libraries: TensorFlow/Keras or PyTorch, NumPy, Pandas, Matplotlib, and scikit-learn.

- Development Environment: Visual Studio Code, Jupyter Notebook, or Google Colab.
- Optional Interface: Flask or Streamlit for a simple web application.

DATASET DESCRIPTION AND PREPARATION

A labelled dataset is required to train and evaluate the model. One commonly used research dataset is the UCI Student Performance dataset, which contains records of secondary-school students from two Portuguese schools, with attributes such as study time, absences, previous failures, family background, and period grades. Dataset selection must be based on the project's target, available attributes, licence, and suitability for the intended use. A dataset should not be described as representing all institutions unless that has been verified.

Dataset Organization

A convenient structure is a single table in which each row represents one student and each column represents an attribute, with one column serving as the target. For example, a project may use study time, absences, previous failures, and first- and second-period grades to predict whether the final grade is a pass. The column names should match the actual dataset and must not be invented after training.

Data Splitting

Divide the dataset into training, validation, and testing subsets. A possible split is 70% training, 15% validation, and 15% testing, although other ratios may be appropriate. The training set updates model parameters, the validation set helps tune hyperparameters and monitor overfitting, and the test set is reserved for final evaluation. Preprocessing statistics, such as scaling values, must be computed on the training set only, because using the whole dataset leaks information and inflates measured performance.

Data Preprocessing

- Handle missing values and remove or correct duplicate and inconsistent records.
- Encode categorical attributes using one-hot encoding or label encoding as appropriate.
- Scale numerical features using standardization or min-max normalization.
- Check for incorrect labels, outliers, and severe class imbalance in the target.
- Apply class weights or resampling only to training data when classes are imbalanced, and avoid using the final-period grade as an input if the goal is early prediction.

METHODOLOGY

Data Collection

Collect records from a public dataset or an institutional database with proper permission. Record the dataset source, version, attribute list, and any usage conditions. Remove personally identifying information, and include enough records per class to support learning rather than relying on a very small number of examples.

Preprocessing and Feature Engineering

Missing values are handled, categorical attributes are encoded, and numerical attributes are scaled so that features with large ranges do not dominate learning. Feature selection or correlation analysis can remove redundant attributes. Any engineered feature, such as an average of earlier assessments, should use only information that would be available at prediction time.

Model Selection

A Deep Neural Network is suitable because stacked hidden layers can learn non-linear combinations of features that simple models may miss. For a tabular dataset of this kind, a fully connected network with two to four hidden layers is a reasonable starting point. Its results should be compared with baseline models such as logistic regression or random forest, so that the benefit of the deep model is measured rather than assumed.

Training

During training, the model predicts a class distribution for each record. A loss function, such as binary or categorical cross-entropy, measures the difference between predicted scores and true labels. An optimizer such as Adam updates model weights through backpropagation. Training should be monitored using validation loss and validation accuracy. Dropout, batch normalization, early stopping, and model checkpointing help reduce overfitting and retain the best validation model.

Prediction

For a new student record, the application performs the same encoding and scaling used during training. The model generates a probability for every supported class, and the highest-scoring class is selected. A confidence score should be presented carefully: a high probability does not necessarily mean the prediction is correct, especially for students whose data differ from the training population.

Output

The application displays the entered student details, the predicted performance category, and an optional confidence score. A responsible implementation should also show a message that the output is an automated prediction and that uncertain or high-impact cases should be reviewed by teachers or counsellors.

SYSTEM ARCHITECTURE

The high-level architecture consists of the following stages:

1. Institution or Dataset → Student Records
2. Data Cleaning and Preprocessing → Encoded, Scaled Feature Vector
3. Trained Deep Neural Network → Class Probabilities
4. Post-processing → Predicted Performance Category
5. User Interface → Prediction, Confidence Estimate, and Advisory Disclaimer

The training pipeline is separate from the prediction pipeline. Training uses labelled records and validation feedback to create a saved model file and a saved preprocessing object. The prediction application loads both and uses them to score new records. This separation makes it easier to update the model without redesigning the interface.

DEEP NEURAL NETWORK MODEL DESIGN

A basic DNN may contain the following conceptual layers. The exact number of neurons and layers should be selected through experiments rather than treated as a universal configuration.

- Input Layer: accepts the preprocessed feature vector, with one unit per encoded feature.
- Dense Hidden Layer 1: a fully connected layer that learns combinations of input features.
- Activation Layer: ReLU introduces non-linearity so the model can learn complex patterns.
- Batch Normalization: stabilizes and speeds up training by normalizing layer outputs.
- Dropout Layer: randomly disables neurons during training to reduce overfitting.
- Additional Hidden Layers: progressively smaller dense layers learn higher-level patterns.
- Output Layer: uses sigmoid for binary prediction, softmax for multi-class prediction, or a linear unit for score prediction.
- Compilation: the model is compiled with a suitable loss function, the Adam optimizer, and evaluation metrics.

For a dataset with K performance classes, the output layer typically contains K units with softmax activation, or a single unit for binary or score prediction. Labels and outputs must use compatible formats, and the same preprocessing object must be applied at training and prediction time.

IMPLEMENTATION PLAN

The project can be implemented in phases so that each part can be tested independently.

1. Phase 1 — Define the prediction target, attributes, dataset, and evaluation plan.
2. Phase 2 — Download or organize the student dataset and inspect it for missing and inconsistent values.
3. Phase 3 — Split the data and build the preprocessing pipeline.
4. Phase 4 — Train a baseline model and a Deep Neural Network.
5. Phase 5 — Evaluate the models on the validation and test sets and inspect errors.
6. Phase 6 — Save the trained model and create a data-entry interface.
7. Phase 7 — Test the full workflow with unseen records and document limitations.

Example Pseudocode

```
Load student performance dataset
Clean data and encode categorical features
Split data into training, validation, and testing sets
Fit scaler on training data and transform all sets
Build DNN with dense, dropout, and output layers
Train model and monitor validation metrics
Save the best model and the scaler
```

```
Evaluate saved model on the held-out test set
For each new student record:
    Validate input values
    Encode and scale the record
    Predict class probabilities using saved model
    Display predicted outcome and suitable disclaimer
```

EVALUATION METRICS

The model should be evaluated on a held-out test set. Accuracy alone can be misleading when classes are imbalanced, since a model that predicts only the majority class may still appear accurate, so class-wise metrics and a confusion matrix should also be reported. If the target is a numerical score, mean absolute error and root mean squared error should be reported instead.

Accuracy

Accuracy = (Number of correct predictions / Total number of predictions) × 100.

Precision

Precision = TP / (TP + FP). Precision indicates how many students predicted to be in a class actually belong to it. Here, TP is true positives and FP is false positives. For an at-risk class, it shows how many flagged students were truly at risk.

Recall

Recall = TP / (TP + FN). Recall indicates how many students who actually belong to a class were correctly identified. FN means false negatives. For early warning, high recall for the at-risk class is important so that struggling students are not missed.

F1-Score

F1-score = 2 × (Precision × Recall) / (Precision + Recall). It balances precision and recall. Macro-averaged metrics give each class equal weight, while weighted averages account for class frequencies.

Confusion Matrix

A confusion matrix compares actual classes with predicted classes. It helps identify which categories the model confuses with one another. For example, students near a pass/fail boundary or between neighbouring grade bands may be misclassified more frequently.

Reporting Results

Do not insert assumed accuracy values into the final report. Train the model and record the actual test results. A results table may be completed as follows:

Metric	Measured result
Test accuracy	_____
Macro precision	_____

Macro recall	_____
Macro F1-score	_____
Number of test records	_____

EXPECTED RESULTS

The expected outcome is a working prototype that can predict the performance category of students from the attributes represented in its training data. The model should learn patterns associated with each outcome and provide a predicted label for a previously unseen student record. Actual performance will depend on dataset size, label quality, class balance, chosen features, model configuration, and the similarity of test data to training data. Results must be measured experimentally and reported honestly.

The project should also produce an evaluation summary, such as accuracy and F1-score, a confusion matrix, a comparison with baseline models, and examples of correct and incorrect predictions. Error analysis is important because it can reveal weaknesses caused by small samples, borderline cases, missing attributes, or under-represented student groups.

ADVANTAGES

- Can provide early identification of students who may need academic support.
- Learns non-linear relationships among features automatically from historical data.
- Can process many student records consistently once trained.
- Can be connected to a simple web interface for teachers and counsellors.
- Can support timely counselling, mentoring, and personalised learning plans.
- Regularization techniques help the model generalize on relatively small datasets.

LIMITATIONS

- Performance depends on dataset size, diversity, and label accuracy.
- Models trained on one institution or year may not generalize to another.
- Important factors such as motivation, health, and personal circumstances may not be recorded in the data.
- Small tabular datasets may not benefit much from deep models over simpler baselines, so comparison is necessary.
- A model confidence score is not a guarantee of correctness.
- Predictions may reflect bias in historical data and must be checked for fairness across student groups.
- Student data is sensitive, so privacy and data protection must be ensured.

These limitations can be reduced—but not completely removed—through diverse data, careful feature selection, fairness checks, uncertainty handling, privacy safeguards, and periodic retraining and monitoring after deployment.

APPLICATIONS

- Early warning systems for identifying at-risk students in schools and colleges.
- Educational demonstrations of deep learning and educational data mining.
- Research prototypes for academic decision-support systems.
- Integration with learning management systems, subject to appropriate validation.
- Support for academic counselling, mentoring, and resource planning.

FUTURE ENHANCEMENTS

- Expand the dataset to cover more subjects, institutions, academic years, and student groups.
- Develop a mobile application that allows teachers to check predictions on the go.
- Use sequential models such as LSTM to learn from semester-wise performance trends over time.
- Add explainability methods such as SHAP or LIME to show which factors influenced each prediction.
- Monitor model performance across different student groups and retrain periodically.
- Provide multilingual instructions and accessible user-interface design.
- Connect predictions to verified support resources, while avoiding automatic decisions without educator review.
- Add fairness and bias audits to ensure predictions do not disadvantage any group.

CONCLUSION

Student Performance Prediction Using Deep Neural Networks demonstrates how deep learning can be applied to an educational problem. By training a DNN on historical student records, a system can learn patterns in academic and behavioural attributes and classify students into supported performance categories. The project combines data preparation, preprocessing, model training, evaluation, and a user-facing prediction interface.

The system can offer a useful first step for early intervention, but its predictions should not be treated as definitive judgements about any student. Reliable deployment requires testing on diverse real-world data, reporting measured performance, checking fairness, protecting privacy, and involving educators in important decisions. With these safeguards, the project provides a practical mini-project for learning deep learning while exploring a meaningful application in education.

REFERENCES

1. Cortez, P., and Silva, A. (2008). Using data mining to predict secondary school student performance. In Proceedings of the 5th Future Business Technology Conference (FUBUTEC 2008), pp. 5–12. https://archive.ics.uci.edu/dataset/320/student+performance
2. Goodfellow, I., Bengio, Y., and Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org/

3. Kingma, D. P., and Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980. https://arxiv.org/abs/1412.6980
4. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research, 15, 1929–1958.
5. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://scikit-learn.org/
6. TensorFlow. Keras guide: training and evaluation, and basic classification tutorials. https://www.tensorflow.org/tutorials/keras/classification